

6.6 Типы виджетов во Flutter

Шаг 1

Во Flutter существует 2 типа виджетов:

- StatelessWidget;
- StatefulWidget.

Шаг 2

StatelessWidget

StatelessWidget – это виджет, который не имеет изменяемого состояния. В конструктор такого виджета передается состояние, которое не изменяется на протяжении его времени жизни. Значит ли это, что их не придется перерисовывать в процессе работы вашего приложения? Нет. Используемые для разработки менеджеры состояния (BLoC, Redux, mobX и т.д.), зачастую используют именно эти виджеты для отображения необходимых данных на пользовательском интерфейсе, беря на себя ответственность отслеживания и изменения состояния приложения, либо его отдельного модуля, после чего запускается процесс создания нового экземпляра класса виджета и его замена на дереве. Говоря другими словами, используя StatelessWidget вы не можете в коде самого виджета описать бизнес-логику и изменить состояние приложения, т.к. для этого вам требуются дополнительные инструменты.

И здесь возникает вполне закономерный вопрос: а что же за зверь такой это состояние? Давайте представим следующую ситуацию. У нас есть виджет «Переключатель», посредством которого мы можем включать и выключать свет в комнате. При использовании StatelessWidget, мы будем в конструктор создаваемого экземпляра класса виджета передавать явный параметр булевского типа, то есть задавать состояние – отобразить для пользователя включен или выключен свет в комнате, без возможности перевести этот переключатель в другое положение средствами самого виджета, не прибегая к сторонней помощи. А StatefulWidget позволяет нам в самом виджете не только хранить начальное состояние переключателя, которое передается ему через конструктора класса, либо используется значение по умолчанию при его инициализации, но и изменять его, вызывая свою перерисовку на графическом пользовательском интерфейсе.

В процессе написания своего виджета с неизменяемым состоянием нам необходимо наследоваться от класса StatelessWidget, переопределив метод build. Для более близкого знакомства с данным типом виджета, создайте новый проект «ex_stateless_widget» или вносите изменения в том, который создали до этого. В качестве примера создадим виджет «MyTextWidget», отвечающий за отображение передаваемой в него текстовой информации, с рядом конфигурируемых параметров, при создании экземпляра класса виджета:



```
// ex_stateless_widget/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyTextWidget extends StatelessWidget {
  final String text;
  final Color textColor;
  final double fontSize;
  const MyTextWidget(
    {super.key,
    required this.text,
    this.textColor = Colors.black,
    this.fontSize = 14});

  @override
  Widget build(BuildContext context) {
    return Text(
      text,
      style: TextStyle(color: textColor, fontSize: fontSize),
    );
  }
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const Scaffold(
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              MyTextWidget(
                text: 'Test1',
              ),
              MyTextWidget(
                text: 'Test2',
                textColor: Colors.indigo,
              ),
              MyTextWidget(
                text: 'Test3',
                textColor: Colors.orange,
                fontSize: 20,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

В конструкторе класса `MyTextWidget` только один обязательный аргумент – «text». Все остальные могут принимать значения по умолчанию. Обратите внимание на то, что конструктор объявлен как константный, это позволяет не создавать каждый

раз новый объект при вызове конструктора класса с одинаковыми параметрами, что в целом сказывается на производительности приложения. Говоря другими словами, экземпляры классов, создаваемые посредством конструктора, вызывают переопределенный метод `build` единожды и так до тех пор, пока не изменятся значения аргументов, передаваемых в константный конструктор класса. Такой конструктор вызывается явным указанием ключевого слова `const`. В нашем случае он используется перед виджетом `Scaffold` и распространяется на его дочерние виджеты.

Переопределение метода `Widget build(BuildContext context)` возвращает компокуемый нами виджет, который добавляется в дерево виджетов, а также является узлом дерева элементов, посредством которого мы можем обращаться к различным свойствам виджетов находящихся на различном уровне его иерархии.

Ниже приведен внешний вид запущенного приложения в эмуляторе:



Рис. 1.33. – Пример создания пользовательского `StatelessWidget`

А теперь давайте зададим себе вопрос: что значит – не может менять свое состояние? Это проще всего пояснить на следующем примере, немного изменив код, генерируемый Flutter при создании нового проекта на:



```
// ex_stateless_widget2/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: MyHomePage(),
    );
  }
}

class MyHomePage extends StatelessWidget {
  int counter;
  String title;

  MyHomePage({
    super.key,
    this.title = 'Flutter Demo Home Page',
    this.counter = 0,
  });

  void _incrementCounter() {
    counter++;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text(
              'You have pushed the button this many times:',
            ),
            Text(
              '$counter',
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

```
}
```

Сколько бы раз вы не нажали на кнопку, значение счетчика на графическом пользовательском интерфейсе не изменится относительно того, что было передано в конструктор виджета. Именно это и значит, что `StatelessWidget` не может менять свое состояние.

Для того, чтобы лучше разобраться с работой константного конструктора модифицируем код следующим образом:



```
// ex_stateless_widget3/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
    debugPrint('Count value = $_counter');
  }

  @override
  Widget build(BuildContext context) {
    debugPrint('Run build MyHomePage widget');
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            MyTextInfo(
              text: 'You have pushed the button this many times:',
            ),
            Text(
              '$_counter',
              style: Theme.of(
                context,
              ).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(

```

```
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}

class MyTextInfo extends StatelessWidget {
  final String text;
  MyTextInfo({super.key, required this.text}) {
    debugPrint('MyTextInfo widget init');
  }

  @override
  Widget build(BuildContext context) {
    debugPrint('Run build MyTextInfo widget');
    return Text(
      text,
    );
  }
}
```

В данном случае мы создали пользовательский виджет, который каждый раз при его создании будет выводить сообщение в терминал (при вызове конструктора и метода build). При нажатии по кнопке у виджета MyHomePage будет изменяться состояние значение счетчика (т.к. это StatefulWidget), после чего вызовется перерисовка всего виджета и в текущем варианте кода виджет MyTextInfo будет пересоздаваться каждый раз:

```
flutter: Run build MyHomePage widget
flutter: MyTextInfo widget init
flutter: Run build MyTextInfo widget
flutter: Count value = 1
flutter: Run build MyHomePage widget
flutter: MyTextInfo widget init
flutter: Run build MyTextInfo widget
flutter: Count value = 2
flutter: Run build MyHomePage widget
flutter: MyTextInfo widget init
flutter: Run build MyTextInfo widget
```



Но что, если у нас таких виджетов много и затрачивать ресурсы системы на их пересоздание не хочется, ведь в них ничего не меняется? Здесь нас и выручит константный конструктор, благодаря которому экземпляр класса виджета MyTextInfo создастся единожды и у него один раз за время существования родительского виджета вызовется метод build. Для этого изменим его конструктор с:

```
MyTextInfo({super.key, required this.text}) {
  debugPrint('MyTextInfo widget init');
}
```



на

```
const MyTextInfo({super.key, required this.text});
```



Тело конструктора с сообщением пришлось убрать, т.к. у константного конструктора его не может быть.

Следующим шагом добавьте ключевое слово const перед создаваемым экземпляром класса MyTextInfo, после чего перезапустите программу:

```
const MyTextInfo(  
  text: 'You have pushed the button this many times:',  
),
```



На выходе терминала можно заметить то, о чем говорилось ранее – экземпляр класса `MyTextInfo` созданся единожды:

```
flutter: Run build MyHomePage widget  
flutter: Run build MyTextInfo widget  
flutter: Count value = 1  
flutter: Run build MyHomePage widget  
flutter: Count value = 2  
flutter: Run build MyHomePage widget  
flutter: Count value = 3  
flutter: Run build MyHomePage widget
```



Согласно документации [5], метод `build` у `StatelessWidget` вызывается в 3-х случаях:

1. При первом добавлении виджета в дерево;
2. При изменении конфигурации родительского виджета;
3. При наличии зависимости от изменений посредством `InheritedWidget`.

Наличие большого количество перестраиваемых элементов в родительском виджете может сказаться на отклике пользовательского интерфейса приложения, поэтому команда Flutter сформировала ряд рекомендаций, которые позволят вам оптимизировать его работу:

1. Насколько возможно минимизируйте вложенность виджетов. Например, вместо сложной компоновки `Rows`, `Columns`, `Paddings` и `SizedBox` для размещения одного дочернего элемента, рассмотрите возможность использования только `Align` или `CustomSingleChildLayout`. Либо вместо сложного наслоения из нескольких виджетов для рисования графического пользовательского интерфейса, рассмотрите возможность использования `CustomPaint`. К этой рекомендации вам придется обращаться очень редко, но лучше об её существовании знать заранее.
2. Создавайте константные виджеты везде, где это только возможно и предоставляйте константный конструктор для ваших пользовательских виджетов.
3. Рассмотрите возможность рефакторинга `StatelessWidget` в `StatefulWidget`, это позволит осуществлять кэширование общих частей поддеревьев и даст возможность использовать `GlobalKeys` при изменении структуры дерева.
4. Если виджет часто перестраивается из-за использования `InheritedWidgets` или менеджеров состояния, рассмотрите возможность рефакторинга `StatelessWidget` на несколько виджетов. При этом части дерева, которые изменяются, перемещайте ближе к листьям, а не его узлам.

Шаг 3

StatefulWidget

При создании виджета, который может менять свое состояние, необходимо наследоваться от `StatefulWidget` и переопределить метод `State<T> createState`, который должен вернуть экземпляр создаваемого виджета, наследуемого от `State<T>`, где `T` – класс, наследующийся от `StatefulWidget`. Звучит немного запутанно, но на самом деле здесь не ничего сложного. Давайте разберем код стартового проекта, который встречается нас каждый раз при создании нового приложения, обратив внимание на процесс создания виджета, который может менять свое состояние:


```
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;
  ...
}
```

Мы объявили класс `MyHomePage`, который наследуется от `StatefulWidget`, принимает на вход конструктора ряд параметров и переопределяет метод `State<T> createState` следующим образом - `State<MyHomePage> createState`. В этом классе будут храниться неизменяемые состояния, в то время как изменяемые объявляются в следующем классе, наследуемом от `State<MyHomePage>`, экземпляр которого и возвращается из переопределенного метода. Такой подход позволяет скрыть саму реализацию виджета, а также предоставляет возможность из класса `_MyHomePageState` получить доступ к полям `MyHomePage`, используя конструкцию вида: `widget.название_поля_класса`.

Класс `State` имеет следующие свойства [5, 6]:

- `context` → `BuildContext` – отвечает за расположение виджета в дереве, в котором он создается;
- `mounted` → `bool` – флаг проверки, находится ли объект в дереве;
- `widget` → `T` – доступ к текущей конфигурации виджета.

Помимо свойств, класс `State` предоставляет разработчику следующие методы для работы с ним:

- `activate()` → `void` – этот метод вызывается в том случае, когда виджет повторно вставляется в дерево после его удаления с помощью метода `deactivate`.
- `build(BuildContext context)` → `Widget` – метод для описания части пользовательского интерфейса, за которую ответственен виджет. Вызывается каждый раз при изменении состояний виджета для его перерисовки. Само изменения состояния может произойти после вызова таких методов, как `initState`, `didChangeDependencies`, `didUpdateWidget`, либо при изменении состояния «вручную» методом `setState`;
- `deactivate()` → `void` – вызывается в случае удаления объекта из дерева, но он еще жив и его можно вставить в другое место дерева до момента обновления кадра;
- `debugFillProperties(DiagnosticPropertiesBuilder properties)` → `void` – позволяет добавлять дополнительные свойства, связанные с текущим узлом в дереве. Используется при отладке.
- `didChangeDependencies()` → `void` – вызывается сразу после `initState` и при изменении состояния объекта посредством `InheritedWidget`. В данном методе также осуществляется подписка на отслеживание изменения состояния в `InheritedWidget`;
- `didUpdateWidget(covariant T oldWidget)` → `void` – вызывается каждый раз при изменении конфигурации виджета. Например, в случае, когда родительский виджет в дереве передает некоторую переменную виджету через конструктор.
- `dispose()` → `void` – этот метод вызывается в случае бесповоротного удаления виджета из дерева и необходим в том случае, если у нас имеются занятые ресурсы (активен контроллер анимации и т.д.), которые необходимо освободить;
- `initState()` → `void` – вызывается при создании экземпляра класса и используется для инициализации изменяемых состояний;
- `reassemble()` → `void` – вызывается при повторной сборке виджета во время отладки. Например, во время запуска `hot reload`.
- `setState(VoidCallback fn)` → `void` – метод для оповещения фреймворка о изменении внутреннего состояния объекта.

С такими методами, как: `reassemble`, `activate`, `debugFillProperties` и прочими (см. документацию) вам нет необходимости знакомиться подробно, но полезно знать о их существовании, поскольку это дает представление о жизненном цикле

виджетов с изменяемым состоянием:

1. Flutter создает объект `State`, вызывая `StatefulWidget.createState`.
2. Созданный объект на постоянной основе связывается с `BuildContext` и его свойство `mounted` устанавливается в `true`. Следует уточнить, что `State` никогда не изменит свой `BuildContext`, но сам `BuildContext` можно перемещать по дереву вместе с его поддеревом.
3. У создаваемого экземпляра класса вызывается переопределяемый метод `initState` для инициализации начальных состояний.
4. Flutter вызывает переопределяемый метод `didChangeDependencies`, который используется для выполнения инициализации с участием `InheritedWidget`. Этот метод также будет вызван снова, если унаследованные виджеты впоследствии изменятся или виджет переместится в дереве.
5. На этом этапе объект `State` полностью инициализирован и Flutter может вызывать метод его сборки любое количество раз, чтобы получить описание пользовательского интерфейса для текущего поддерева. При этом объекты `State` могут спонтанно запрашивать перестройку своего поддерева, вызывая метод `setState`, который указывает, что произошло изменение некоторых внутренних состояний.
6. На текущей стадии жизненного цикла родительский виджет может перестроиться и запросить обновление дерева для отображения нового виджета с тем же `runtimeType` и `Widget.key`. Когда это происходит, Flutter обновляет свойство виджета, чтобы ссылаться на новый виджет, после чего вызывает метод `didUpdateWidget` с предыдущим виджетом в качестве аргумента. Пользовательские `StatefulWidget` должны переопределять метод `didUpdateWidget` для ответа на изменения в связанном с ними виджете (например, для запуска неявной анимации). После вызова `didUpdateWidget` всегда вызывается метод `build`, вследствие чего вызовы `setState` в `didUpdateWidget` являются избыточными.
7. Если во время разработки приложения происходит `hot reload` это приводит к вызову метода повторной сборки – `reassemble`, что дает возможность повторно инициализировать любые в `initState`.
8. Если происходит удаление поддерева, которое содержит объект `State`, то Flutter вызывает метод `deactivate`. Пользовательский класс должен переопределять этот метод, когда необходимо очистить любые связи между экземпляром класса (объектом) и другими элементами в дереве (например, если был предоставлен базовому классу указатель на производный).
9. Пока поддерево окончательно не удалено его можно повторно вставить в другую часть дерева. Когда это происходит, Flutter вызывает метод `build` для адаптации объекта `State` к его новому положению в дереве. Flutter должен вставить это поддерево в рамках текущего кадра, то есть до начала отрисовки нового. Именно по этой причине объекты `State` могут отложить высвобождение большинства ресурсов до тех пор, пока Flutter не вызовет их метод `dispose`.
10. Если Flutter повторно не вставит это поддерево к концу текущего кадра, то у объекта `State` вызовется метод `dispose` и он никогда не будет создан снова. Ваши пользовательские классы должны переопределить этот метод для освобождения любых ресурсов, которые могут удерживаться удаляемым объектом.
11. После вызова метода `dispose` объект `State` считается отключенным, а его свойство `mounted` устанавливается в `false`. Эта последняя стадия жизненного цикла, на которой происходит удаление объекта, из-за чего нет возможности смонтировать его повторно.

В графическом виде жизненный цикл `State` может быть представлен следующим образом:

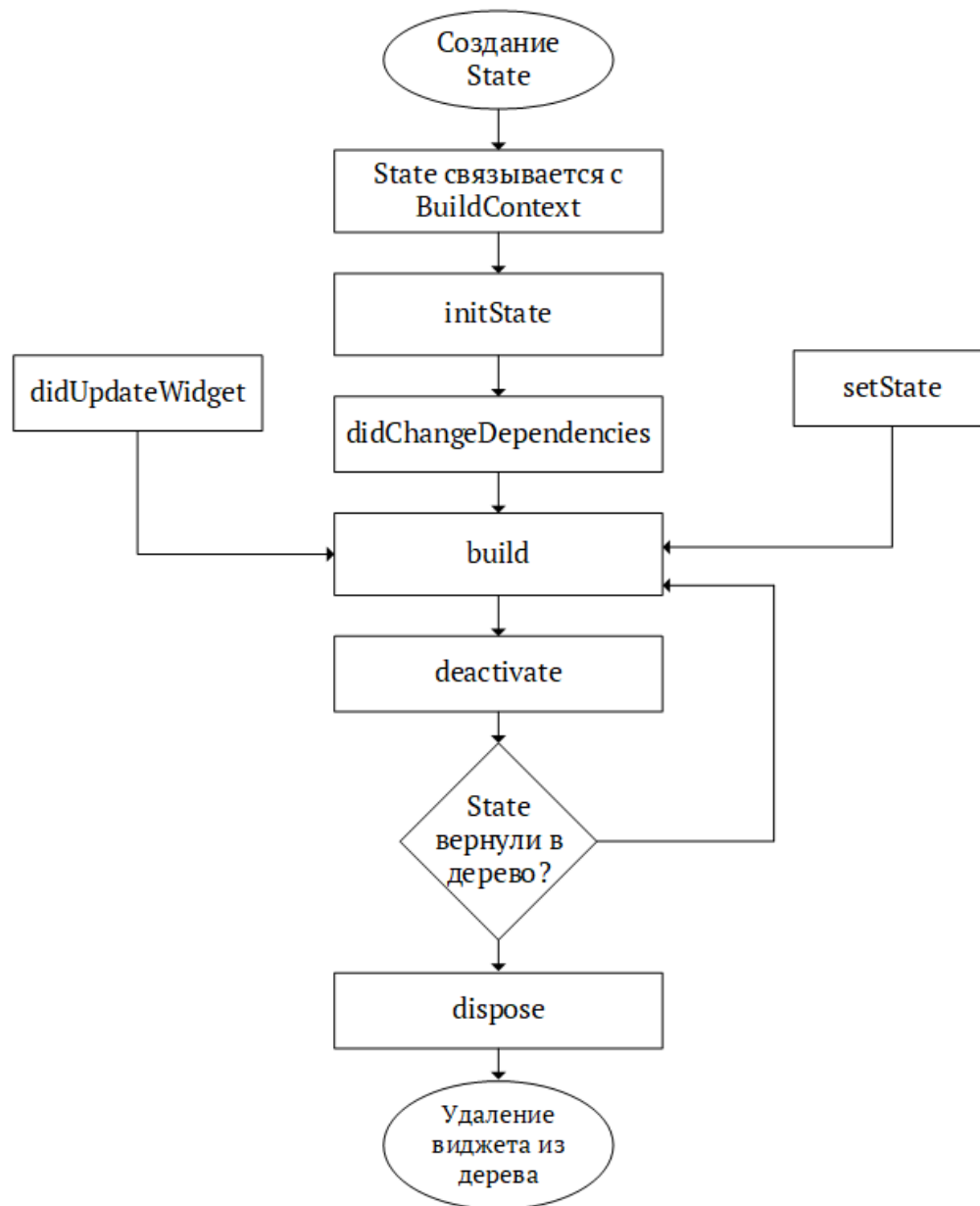


Рис. 1.34. – Жизненный цикл State

В качестве примера демонстрации возможностей StatefulWidget, давайте реализуем следующее приложение. На главном экране пользователю будет доступна одна единственная кнопка, с текстом «». По ее нажатию, пользователя переносит на следующий экран (с возможностью возврата), где его ожидает StatefulWidget виджет с привычной реализация увеличения значения счетчика по нажатию на кнопку. С тем отличием, что она находится посередине, а в правом нижнем углу расположим кнопку, по нажатию которой будет выполняться оборачивание центрального виджета в цветной контейнер и запуск его перерисовки. Повторное нажатие, должно же убрать контейнер и снова запустить процесс обновления графического пользовательского интерфейса. А для отслеживания изменений жизненного цикла StatefulWidget, осуществим переопределение некоторых из рассмотренных ранее методов.

Создайте новый проект «ex_stateful_widget1» со следующим начальным наполнением:

```
// ex_stateful_widget1/lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ),
        useMaterial3: true,
      ),
      home: const StartPage(),
    );
  }
}

class StartPage extends StatelessWidget {
  const StartPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('StatefulWidget LifeCircle'),
      ),
      body: Center(
        child: FloatingActionButton.extended(
          onPressed: () { // при нажатии на кнопку
            Navigator.push( // переход на новую страницу
              context,
              MaterialPageRoute(builder: (context) {
                return Scaffold(
                  appBar: AppBar(
                    title: const Text('MainPage'),
                  ),
                  body: const Center(
                    child: Text(
                      'Hello World!',
                      textAlign: TextAlign.center,
                      style: TextStyle(fontSize: 24.0),
                    ),
                  ),
                );
              })),
            );
          },
          label: const Text('Go MainPage')),
        ),
      );
  }
}
```

Если осуществить сборку проекта, то пользователя ждет кнопка в центре главного окна, нажатие которой переведет его на следующий экран, из которого можно вернуться обратно:

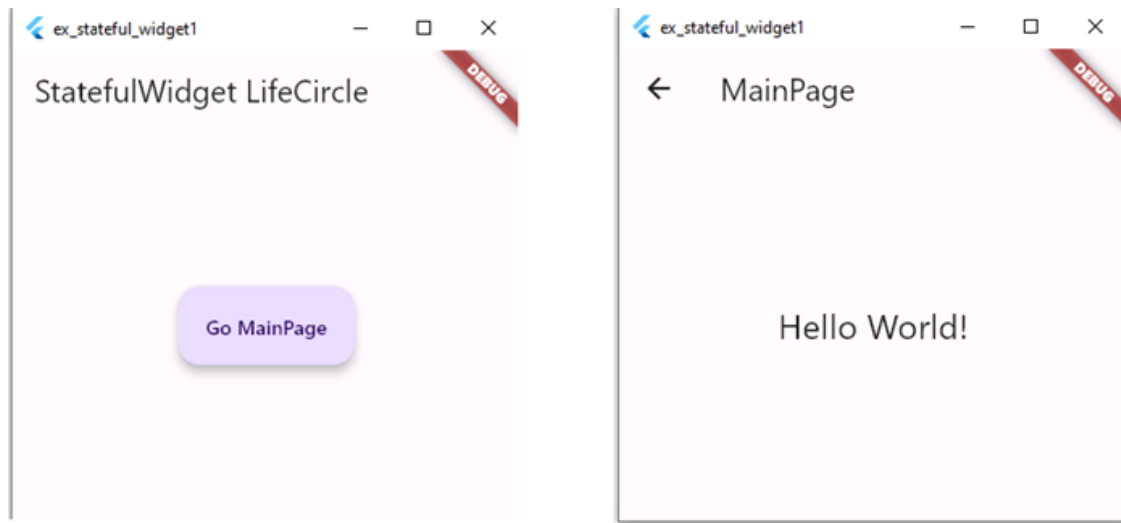


Рис. 1.35. – Переход между экранами приложения

Далее добавим в «main.dart» StatefulWidget, который будет содержать Scaffold, располагаться следующем экране и менять значение переменной, отвечающей за то, надо ли оборачивать виджет со счетчиком в контейнер или нет:

```
class _MainPageState extends State<MainPage> {
  bool _isWrapNewPage = false;

  // Метод для переключения значения _isWrapNewPage и обновления UI.
  void wrapNewPage() {
    // _isWrapNewPage != _isWrapNewPage;
    _isWrapNewPage = _isWrapNewPage ? false : true;
    setState(() {});
    debugPrint('isWrapNewPage: $_isWrapNewPage');
  }

  @override
  Widget build(BuildContext context) {
    debugPrint('MainPage build');
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: MyStatefulWidget(
          isWrapped: _isWrapNewPage,
        )),
      floatingActionButton: FloatingActionButton(
        onPressed: wrapNewPage,
        child: const Icon(Icons.toc),
      ),
    );
  }
}
```

Наличие функций debugPrint позволит отследить каждую перерисовку виджета и его изменение состояний, выводя оповещения в терминал. А чтобы добавить MainPage в приложение, замените код в Navigator.push с:

```
Navigator.push( // переход на новую страницу
  context,
  MaterialPageRoute(builder: (context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('MainPage'),
      ),
      body: const Center(
        child: Text(
          'Hello World!',
          textAlign: TextAlign.center,
          style: TextStyle(fontSize: 24.0),
        ),
      ),
    );
  });
```

на

```
Navigator.push( // переход на новую страницу
  context,
  MaterialPageRoute(builder: (context) {
    return MainPage(title: 'Main StatefulWidget');
  })),
);
```

Теперь, после нажатия на кнопку «Go MainPage» изменится экран с виджетом, на который будет осуществлен переход:

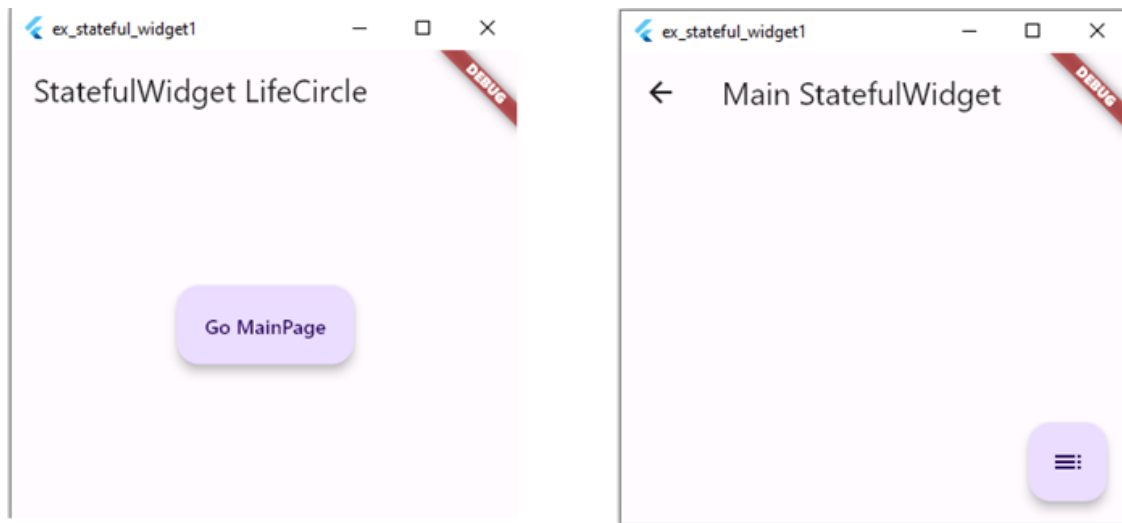


Рис. 1.36. – Переход между экранами приложения

Следующим действием, реализуем класс `MyStatefulWidget` со счетчиком и переопределим ряд методов, чтобы можно было отследить передвижение по жизненному циклу `StatefulWidget`:



```
class MyStatefulWidget extends StatefulWidget {
  final bool isWrapped;
  const MyStatefulWidget({super.key, required this.isWrapped});

  @override
  State<MyStatefulWidget> createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  int _counter = 0;
  late bool _isWrapped;

  // Вызывается при создании экземпляра класса и
  // используется для инициализации изменяемых состояний
  @override
  void initState() {
    super.initState();
    _isWrapped = widget.isWrapped;
    debugPrint('--MyStatefulWidget initState');
  }

  // Вызывается сразу после initState и при изменении
  // состояния объекта посредством InheritedWidget
  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    debugPrint('--MyStatefulWidget didChangeDependencies');
  }

  // Вызывается каждый раз при изменении конфигурации виджета
  @override
  void didUpdateWidget(covariant MyStatefulWidget oldWidget) {
    super.didUpdateWidget(oldWidget);
    debugPrint('--MyStatefulWidget didUpdateWidget');
    if (oldWidget.isWrapped == widget.isWrapped) return;
    _isWrapped = widget.isWrapped; // обновляем состояние
  }

  // Вызывается в случае удаления объекта из дерева,
  // но он еще жив и его можно вставить в другое место дерева
  // до момента обновления кадра
  @override
  void deactivate() {
    super.deactivate();
    debugPrint('--MyStatefulWidget deactivate');
  }

  // Метод освобождения ресурсов, используемых MyStatefulWidget.
  // Вызывается в случае бесповоротного удаления виджета из дерева
  @override
  void dispose() {
    debugPrint('--MyStatefulWidget dispose');
    super.dispose();
  }

  // Пересобирает состояние виджета при Hot Reload.
  // Вызывает метод пересборки суперкласса, устанавливает
  // состояние с новым значением для переменной _counter и
  // выводит отладочное сообщение, указывающее на горячую
  // перезагрузку MyStatefulWidget.
  @override
  void reassemble() {
    super.reassemble();
    setState(() {
      _counter = 32;
    });
    debugPrint('--MyStatefulWidget hot reload');
  }
}
```

```

}

// Увеличивает и выводит в терминал значение счетчика,
// а также обновляет состояние виджета.
void _incrementCounter() {
  setState(() {
    _counter++;
  });
  debugPrint('--Counter: $_counter');
}

// Создает виджет на основе значения _isWrapped.
// Если _isWrapped равно true, возвращает контейнер
// с полупрозрачным оранжевым цветом и результатом
// _buildWidget в качестве дочернего элемента.
// В противном случае возвращает результат выполнения _buildWidget.
@override
Widget build(BuildContext context) {
  debugPrint('--MyStatefulWidget build with wrap is: $_isWrapped');
  return _isWrapped
    ? Container(
        color: Colors.deepOrange.withOpacity(0.5),
        child: _buildWidget(),
      )
    : _buildWidget();
}

// Создает и возвращает виджет, содержащий колонку
// с виджетом текста, отображающую текущее значение счетчика,
// и ElevatedButton, при нажатии на которую вызывается функция
// отвечающая за увеличение значения счетчика.
Widget _buildWidget() {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Text(
        '$_counter',
        style: Theme.of(context).textTheme.headlineMedium,
      ),
      ElevatedButton(
        onPressed: () {
          _incrementCounter();
        },
        child: const Text('Increment'))
    ],
  );
}
}

```

Чтобы добавить данный виджет в приложение, замените аргумент body виджета Scaffold у класса _MainPageState с:

```
body: const Center(),
```

на

```
Center(
  child: MyStatefulWidget(
    isWrapped: _isWrapNewPage,
  )),
```

По нажатию на кнопку «Increment» значение счетчика увеличится на единицу, а на правую нижнюю кнопку, центральный виджет будет оборачиваться в цветной контейнер. Повторное нажатие вернет все на место:

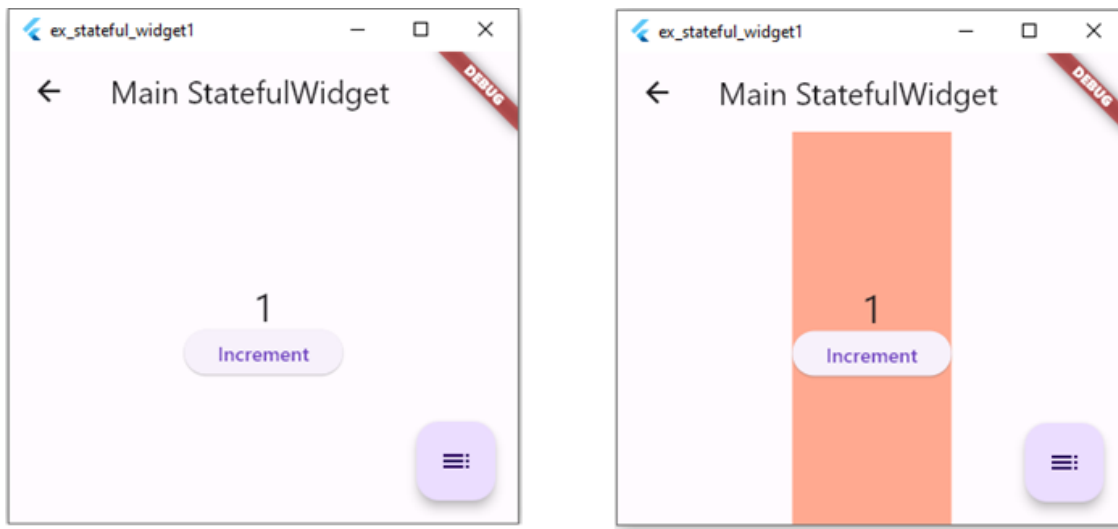


Рис. 1.37. – Оборачивание центрального виджета в контейнер

Любое перехваченное действие в процессе нажатий на кнопки и перерисовок, отобразиться в терминале:

```
flutter: MainPage constructor
flutter: MainPage build
flutter: --MyStatefulWidget initState
flutter: --MyStatefulWidget didChangeDependencies
flutter: --MyStatefulWidget build with wrap is: false
flutter: --Counter: 1
flutter: --MyStatefulWidget build with wrap is: false
flutter: --Counter: 2
flutter: --MyStatefulWidget build with wrap is: false
flutter: isWrapNewPage: true
flutter: MainPage build
flutter: --MyStatefulWidget didUpdateWidget
flutter: --MyStatefulWidget build with wrap is: true
flutter: isWrapNewPage: false
flutter: MainPage build
flutter: --MyStatefulWidget didUpdateWidget
flutter: --MyStatefulWidget build with wrap is: false
flutter: --Counter: 3
flutter: --MyStatefulWidget build with wrap is: false
flutter: isWrapNewPage: true
flutter: MainPage build
flutter: --MyStatefulWidget didUpdateWidget
flutter: --MyStatefulWidget build with wrap is: true
```

А если при запущенном приложении, измените в коде название кнопки с «Increment» на «Increment1» и сохраните эти изменения, запустив Hot Reload, то, в соответствии с переопределенным методом `reassemble`, значение счетчика установится в 32:

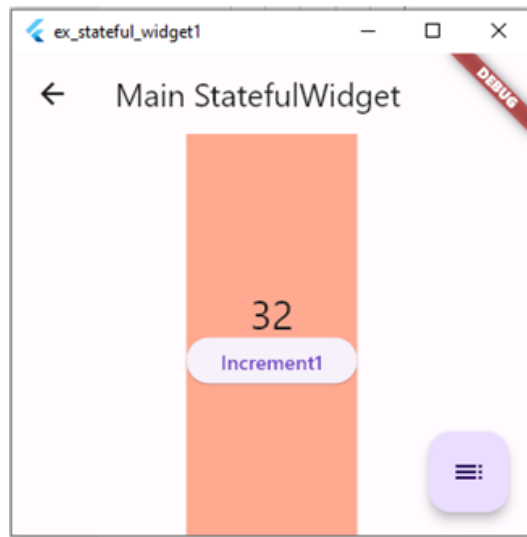


Рис. 1.38. – Результат работы метода `reassemble` в процессе Hot Reload

Для более лучшего понимания процессов, происходящих в приложении по нажатию на кнопки и смену экранов (главный на следующий и обратно), обращайте внимание на вывод состояний в терминал. К тому же, такое «исследование» позволит лучше проникнуться философией жизненного цикла `StatefulWidget`.

Шаг 4

Что из перечисленного является ключевой особенностью `StatelessWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/4>

Шаг 5

Какой метод обязательно должен быть переопределен в классе, унаследованном от `StatelessWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/5>

Шаг 6

Какая из ситуаций наиболее подходит для использования `StatelessWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/6>

Шаг 7

Что произойдёт, если родительский виджет изменит параметры, передаваемые в конструктор `StatelessWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/7>

Шаг 8

Ниже приведены различные варианты объявления `StatelessWidget` . Какой из них правильный?

```
// A
class MyWidget extends StatelessWidget {
  Widget render(BuildContext context) {
    return Text('Hello');
  }
}
```



```
// Б
class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
```



```
// B
class MyWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
```



```
// Г
class MyWidget extends StatefulWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
```



Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/8>

Шаг 9

Какие свойства присущи классу `State`?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/9>

Шаг 10

В чем основное отличие `StatefulWidget` от `StatelessWidget` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/10>

Шаг 11

Какой метод необходимо переопределить в производном классе от `StatefulWidget` , чтобы связать его с состоянием?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/11>

Шаг 12

Что делает метод `setState()` внутри класса `State<T>` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/12>

Шаг 13

Ниже приведены различные варианты объявления `StatefulWidget` . Какой из них правильный?

```
// A
class MyWidget extends StatefulWidget {
  @override
  _MyWidgetState createState() => _MyWidgetState();
}
class _MyWidgetState extends State<MyWidget> {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}

// Б
class MyWidget extends StatefulWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
class _MyWidgetState extends State<MyWidget> {}

// В
class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
class _MyWidgetState extends State<MyWidget> {}

// Г
class MyWidget extends StatefulWidget {
  @override
  void initState() {}
  @override
  Widget build(BuildContext context) {
    return Text('Hello');
  }
}
```

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/13>

Шаг 14

Какой метод класса `State` вызывается единожды при инициализации и до первого вызова `build()` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/14>

Шаг 15

Что из перечисленного верно описывает метод `didChangeDependencies()` ?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/15>

Шаг 16

Какой метод следует использовать для освобождения ресурсов (например, отмены таймеров или удаления слушателей) перед удалением `State` из дерева виджетов?

Чтобы решить это задание откройте <https://stepik.org/lesson/1242585/step/16>